Unit 1: Introduction (5 Hrs)

This unit lays the foundation. The key is to understand the 'why' behind DBMS, not just the 'what'.

1. Concepts of Data, Database, and DBMS

- **Data:**
  Raw, unprocessed facts and figures that have no meaning by themselves (e.g., "Ram," "25," "Kathmandu").
  - **Real-World Analogy:** Individual Lego bricks scattered on a floor.

- **Database:**
  An organized collection of related data, designed for a specific purpose. It's a structured repository.
  - **Real-World Analogy:** The assembled Lego castle built from the bricks, where each brick has a specific place and purpose.

- **DBMS (Database Management System):**
  A software system that enables users to define, create, maintain, and control access to the database. It acts as an intermediary between the database and its users/applications.
  - **Real-World Analogy:** The instruction manual and the process that guides you on how to assemble the Lego bricks into a castle, ensuring the structure is sound and you can find/access any brick you need.
  - **Examples:** MySQL, Oracle, PostgreSQL, Microsoft SQL Server.

2. Need for Having a DBMS / Purpose of Database System Over Traditional File Systems

The traditional file system (like keeping data in spreadsheets or text files) was prone to serious problems. A DBMS solves them .

- **Problem: Data Redundancy and Inconsistency**

  - **File System:** Same data duplicated in multiple files (e.g., student address in both library and hostel files). Wastes space and leads to **inconsistency** (different copies don't match).
  - **DBMS:** Redundancy is **controlled and minimized**. Data is stored once and linked, ensuring consistency.

- **Problem: Difficult Data Access**

  - **File System:** Generating a report ("Find all students in 'Database' course") might require writing a new program.
  - **DBMS:** Uses a powerful **query language (SQL)** for flexible, ad-hoc queries, answered instantly.

- **Problem: Data Isolation**
  - **File System:** Data is in disparate files with different formats, making it hard to write new programs to access related data.
  - **DBMS:** Provides a **centralized, unified view** of all data, making it easy to access and relate different data elements.

- **Problem: Integrity Problems**
  - **File System:** Rules (e.g., "age must be >= 18") are enforced by application code, which is easy to bypass or forget.
  - **DBMS:** Enforces **integrity constraints** (e.g., CHECK constraint) at the database level, ensuring data accuracy regardless of the application.

- **Problem: Atomicity Problems**
  - **File System:** If a system fails during an operation (e.g., transferring money), the database is left in an inconsistent state (money debited but not credited).
  - **DBMS:** Ensures **Atomicity** - a transaction is treated as a single, indivisible unit. It either happens completely or not at all.

- **Problem: Concurrent Access Anomalies**
  - **File System:** If two users try to book the last seat simultaneously, the system can corrupt the data.
  - **DBMS:** Manages **concurrent access** through transactions and locking, ensuring data integrity, like modern ticket booking systems.

- **Problem: Security Problems**
  - **File System:** Security is coarse-grained (file-level permissions).
  - **DBMS:** Offers **fine-grained security**, allowing access control at the table, row, or even column level for different users.

Advantages of DBMS :

- **Data Independence:** Application programs are independent of how data is stored.
- **Efficient Data Access:** Uses sophisticated techniques to store and retrieve data efficiently.
- **Data Integrity and Security:** Centralized control ensures data accuracy and protects against unauthorized access.
- **Data Administration:** Centralized management of data resources.
- **Resilient Concurrency and Crash Recovery:** Handles multiple users and recovers from failures.

Disadvantages/Limitations of DBMS :

- **Cost:** High initial cost for software, hardware, and expertise.
- **Complexity:** DBMS is complex software; administrators need significant training.

- **Performance Overhead:** May have slower processing for simple, single-user applications compared to file systems.
- **Higher Impact of Failure:** Centralization means a database failure can halt entire organization's operations.

3. Levels of Abstraction (Three-Schema Architecture)

Data abstraction is the process of hiding irrelevant details from the user to simplify interaction with the database . There are three levels:

- **1. Physical Level (Internal Level):**
  The lowest level of abstraction.
  - **What it describes: HOW** the data is actually stored. It deals with complex low-level data structures (file paths, indexes, bytes, blocks).
  - **Who cares about it:** Database Administrators (DBAs) and system developers.
  - **Real-World Analogy:** The actual physical arrangement of books in a library, which shelf, which row, the material of the book, and the ink used to print the words.

- **2. Logical Level (Conceptual Level):**
  The next higher level of abstraction.
  - **What it describes: WHAT** data is stored and the **relationships** among the data. It describes the entire database in terms of a simple structure (e.g., tables, columns, data types, constraints). Users at this level are shielded from physical storage details.
  - **Who cares about it:** Database Designers and Administrators.
  - **Real-World Analogy:** The library's card catalog (or digital catalog). It describes the books by their title, author, ISBN, and subjects, but doesn't tell you which physical shelf they are on.

- **3. View Level (External Level):**
  The highest level of abstraction.
  - **What it describes:** Only a **part** of the entire database. It provides a customized view for different categories of users, showing only the data they need and hiding the rest.
  - **Who cares about it:** End-users and application programs.
  - **Real-World Analogy:** A library user searching the online portal only sees the book's title, availability, and call number. They don't see the entire database schema used by the librarian.

Data Independence: The main advantage of this three-level architecture is that it allows changes at one level without affecting the higher levels .

- **Logical Data Independence:** The ability to change the logical schema (e.g., add a new column to a table) without affecting external views or application programs.
- **Physical Data Independence:** The ability to change the physical schema (e.g., create a new index for faster access) without changing the logical schema.

4. Increasing Trends of Storage Space Required

- **Drivers of Growth:**
    - **Digitization of Everything:** From government records and healthcare to social media and IoT sensors, everything is generating data.
    - **Multimedia Content:** High-definition videos, images, and audio files consume massive storage.
    - **Big Data and Analytics:** Organizations are storing petabytes of data for business intelligence and machine learning.
- **Implication:** This exponential growth is the primary reason we need robust, scalable, and efficient DBMS. A simple file system cannot handle the volume, velocity, and variety of modern data.

Unit 2: Relational Algebra (10 Hrs)

This unit is crucial for understanding how queries are logically executed. Relational algebra is the procedural language for relational databases .

1. Basic Concepts

- **Relational Model:** Data is organized into tables called **relations**.
- **Relation:** A table with rows and columns.
- **Tuple:** A row in a relation.
- **Attribute:** A column in a relation.
- **Domain:** A set of allowable values for an attribute (e.g., integer, varchar(50)).
- **Relational Algebra:** A set of operations that take one or two relations as input and produce a new relation as output. This property is called **closure** .
- **Relational Completeness:** A data manipulation language (like SQL) is relationally complete if it is at least as powerful as relational algebra .

2. DDL & DML

- **DDL (Data Definition Language):**
  Used to define the database structure/schema.
    - **Commands:** CREATE, ALTER, DROP, TRUNCATE, RENAME.
    - **Example:** CREATE TABLE Student (id INT PRIMARY KEY, name VARCHAR(50));
- **DML (Data Manipulation Language):**
  Used to manipulate data within the schema.
    - **Commands:** SELECT, INSERT, UPDATE, DELETE.
    - **Example:** INSERT INTO Student VALUES (1, 'Ram');

3. Fundamental Relational-Algebra-Operations (Unary)

These are the essential, irreducible operations .

- **1. Select (σ):**
  - **Symbol:** $\sigma_{predicate}$(Relation)
  - **Function:** Selects **rows (tuples)** that satisfy a given predicate (condition). It's a horizontal filter.
  - **Example:**
    Find all students from "Kathmandu".
    - σ_{city = "Kathmandu"}(Student)

- **2. Project (π):**
  - **Symbol:** $\pi_{attribute1,\ attribute2,...}$(Relation)
  - **Function:** Selects specific **columns (attributes)**. It's a vertical filter. It automatically removes duplicate rows from the result.
  - **Example:**
    Get only the names and ages of all students.
    - π_{name, age}(Student)

- **3. Rename (ρ):**
  - **Symbol:** $\rho_{new_name(attribute1,...)}$(Relation) or ρ_{new_name}(Relation)
  - **Function:** Returns the same relation but with a new name, or with renamed attributes. It's essential for self-joins and clarity.
  - **Example:**
    Rename
    Student
    table as
    S
    .
    - ρ_S(Student)

4. Fundamental Relational-Algebra-Operations (Binary Set Operations)

These operations require the input relations to be **union compatible** .

- **Union Compatible:** Both relations must have the same number of attributes, and the corresponding attributes must have the same data type.

- **4. Union (□):**
  - **Function:** Returns a relation containing all tuples that are in **either** of the two specified relations. Duplicates are eliminated.
  - **Example:**
    Find all cities where either a student or a professor lives.
    - π_{city}(Student) □ π_{city}(Professor)

- **5. Set Difference (-):**
  - **Function:** Returns a relation containing all tuples that are in the **first** relation but **not** in the second.
  - **Example:**
    Find students who have not registered for any course.
    - π_{student_id}(Student) - π_{student_id}(Registration)

- **6. Cartesian Product (×):**
  - **Function:** Combines every tuple of the first relation with every tuple of the second relation. It can create a very large relation.
  - **Example:**
    Combine
    Student
    and
    Course
    tables.
    - Student × Course

5. Additional Relational-Algebra-Operations

These operations don't add new theoretical power but are extremely useful and common .

- **7. Set Intersection (□):**
  - **Function:** Returns a relation containing all tuples that are in **both** of the specified relations.
  - **Example:**
    Find cities where both a student and a professor live.
    - π_{city}(Student) □ π_{city}(Professor)

- **8. Join (□):**
  - **Function:** Allows us to combine related tuples from two relations into a single tuple. It's essentially a Cartesian product followed by a selection.

- **Types:**
  - **Theta Join ($\bowtie_\theta$):** A join with a general condition (e.g., Student ⋈ _{Student.id= Reg.s_id} Reg).
  - **Equi-Join:** A theta join where the condition is an equality (=). It often results in two identical columns (the join attributes).
  - **Natural Join (*):**
    An equi-join on
    all
    attributes with the same name, and one of the duplicate columns is eliminated.
    - **Example:** Assuming Student and Registration both have an id column, the natural join automatically joins on id and removes the duplicate.

- **9. Division (÷):**

  - **Function:** Used for queries that involve "for all" or "all of". It finds tuples in one relation that match **all** tuples in another relation.
  - **Real-World Example:**
    "Find the suppliers who supply
    all
    parts."
    - π_{supplier_id, part_id}(Supplies) ÷ π_{part_id}(Parts)

  - **Analogy:** It's like having a set of skills required for a job and a set of skills each person has. Division finds people who have all the required skills.

6. Extended Relational-Algebra-Operations

These operations extend the power of relational algebra for real-world tasks, particularly aggregation.

- **10. Generalized Projection:**
  Allows arithmetic operations in the projection list.
  - **Example:** π_{name, salary * 1.1}(Employee) to project names and a 10% increased salary.

- **11. Aggregate Functions and Grouping (𝒢):**

  - **Symbol:** $_{grouping_attributes}\mathcal{G}_{function_list}$ (Relation)
  - **Function:** Performs calculations on a set of values to return a single summarized value.
  - **Common Functions:** COUNT, SUM, AVG, MIN, MAX.
  - **Example:**
    Find the average salary per department.
    - $_{dept_name}\mathcal{G}_{AVG(salary)}$(Employee)

7. Null Values

- **Definition:** NULL represents a value that is **unknown**, **missing**, or **not applicable**.
- **Challenges:**
  - Arithmetic operations involving NULL result in NULL (e.g., 5 + NULL = NULL).
  - Comparisons involving NULL yield a special truth value: **UNKNOWN**.
  - Logical connectands (AND, OR, NOT) must now operate with three-valued logic (True, False, Unknown).
- **Handling:** In relational algebra, we have special conditions like IS NULL and IS NOT NULL to check for null values.

8. Modification of the Database

While relational algebra is primarily for querying, its operations form the basis for modifications:

- **Deletion:**
  A
  DELETE
  operation can be seen as a
  Set Difference
  .
  - r ⊟ r - σ_{condition}(r)
- **Insertion:**
  An
  INSERT
  operation can be seen as a
  Union
  .
  - r ⊟ r ⊟ E (where E is a set of new tuples to insert)
- **Updating:** An UPDATE can be modeled using a combination of **Projection, Rename, and Join**, though it's more complex. It's essentially a deletion of the old tuple and an insertion of the modified tuple.

Exam Tips for Units 1 & 2

- **Unit 1:** Be prepared to write long answers on the "Purpose of DBMS over File System." Use the table format to contrast them in your answer for a high-impact start. For "Levels of Abstraction," always use the library analogy to explain physical, logical, and view levels—it makes the answer memorable .
- **Unit 2:**

This is a problem-solving unit. You
must
practice writing relational algebra expressions.

- Remember the **closure** property: the output of one operation is the input for another .
- For division problems (like "all suppliers who supply all parts"), remember the standard 5-step approach.
- For exam queries, always specify which operators you are using (σ, π, ▯ etc.) clearly.

Unit 3: SQL / Advanced SQL (10 Hrs)

SQL is the practical language of databases. This unit moves from basic queries to powerful advanced features that professionals use daily.

1. General Concepts and Basic Structure of SQL Queries

- **SQL (Structured Query Language):** A non-procedural language used to communicate with relational databases. It's the standard language for defining, manipulating, and controlling data .
- **Basic Query Structure:** sql SELECT column1, column2 -- Projection (π): which columns to show FROM table_name -- which table(s) to get data from WHERE condition; -- Selection (σ): which rows to filter
- **Query Execution Order (Conceptual):** FROM ▯ WHERE ▯ GROUP BY ▯ HAVING ▯ SELECT ▯ ORDER BY. Understanding this helps in debugging complex queries.
- **Real-World Analogy:** Think of SQL as ordering food at a restaurant. SELECT is "I want a burger and fries," FROM is "from the menu," WHERE is "with no onions" – you specify what you want, not how the kitchen prepares it.

2. SQL Data Types & Schemas

- **Numeric Types:** INT, SMALLINT, DECIMAL(p,s) (fixed-point), FLOAT, REAL.
- **String Types:** CHAR(n) (fixed-length), VARCHAR(n) (variable-length), TEXT for large strings.
- **Date/Time Types:** DATE, TIME, TIMESTAMP, INTERVAL.
- **Boolean:** TRUE, FALSE, UNKNOWN.
- **Large Object Types:** BLOB (Binary Large Object for images/files), CLOB (Character Large Object for text).
- **JSON/XML Types:** Modern databases support native JSON and XML data types for semi-structured data .
- **Schema:** A named collection of tables, views, privileges, and other database objects. It's like a namespace (e.g., "UniversitySchema.Student").

3. Integrity Constraints

Integrity constraints are rules that ensure data accuracy and consistency . They are the

database's way of enforcing business rules automatically.

Constraint	Purpose	Real-World Analogy	Example
NOT NULL	Ensures a column cannot have NULL (empty) values .	Every employee **must** have a name on their ID card.	name VARCHAR(100) NOT NULL
UNIQUE	Ensures all values in a column (or set) are distinct .	No two citizens can have the same passport number.	email VARCHAR(100) UNIQUE
PRIMARY KEY	Uniquely identifies each row. Combines NOT NULL + UNIQUE .	A university's roll number uniquely identifies one student.	student_id INT PRIMARY KEY
FOREIGN KEY	Links two tables. Ensures values in one table match values in another .	An order must belong to a customer that actually exists.	FOREIGN KEY (dept_id) REFERENCES Department(id)
CHECK	Validates data based on a logical expression .	A person's age cannot be negative.	CHECK (age >= 18)
DEFAULT	Provides a default value when none is specified .	If no hire date is provided, use today's date.	hire_date DATE DEFAULT CURRENT_DATE

Referential Integrity Actions (with FOREIGN KEY):

- ON DELETE CASCADE: If a parent row is deleted, delete all child rows referencing it.
- ON DELETE SET NULL: If a parent row is deleted, set the foreign key in child rows to NULL.
- ON DELETE RESTRICT/NO ACTION: Prevent deletion if child rows exist.

4. Advanced SQL Features

These features separate beginners from professionals. They allow complex data manipulation and analysis directly in SQL .

A. Joins (Beyond Basic Inner Join)

- **LEFT JOIN (or LEFT OUTER JOIN):**
  Returns all rows from the left table, and matched rows from the right. If no match, NULLs appear for right table columns.
  - **Use Case:** "Show me all students and their enrollment dates, even if they haven't enrolled in anything."

- **RIGHT JOIN:** Opposite of LEFT JOIN. All rows from the right table.
- **FULL OUTER JOIN:** Returns all rows from both tables, matching where possible, with NULLs elsewhere.
- **SELF JOIN:**
  Joining a table to itself (using table aliases).

- **Use Case:** "Find employees and their managers (both stored in the same Employee table)."

B. Subqueries and Nested Queries

- Queries inside queries. Can be used in SELECT, FROM, or WHERE clauses.
- **Scalar Subquery:**
  Returns a single value.
  - SELECT name, (SELECT COUNT(*) FROM Orders WHERE customer_id = C.id) AS order_count FROM Customer C;

- **Correlated Subquery:** Inner query references outer query. Executes row-by-row.
- **Use Case:** Finding employees earning more than the average salary of their own department.

C. Aggregate Functions and GROUP BY with HAVING

- **Aggregate Functions:** COUNT(), SUM(), AVG(), MIN(), MAX(). They collapse multiple rows into a single result.
- **GROUP BY:** Groups rows that have the same values in specified columns, allowing aggregates to be applied per group.
- **HAVING:** Filters groups based on a condition (like WHERE, but for groups after aggregation) .
- **Example:** Find departments with average salary > 50,000. sql SELECT dept_id, AVG(salary) AS avg_sal FROM Employee GROUP BY dept_id HAVING AVG(salary) > 50000;

D. Window Functions

Window functions perform calculations across a set of rows related to the current row **without collapsing them** . Each row retains its identity.

- **Key Concept:**

  OVER (PARTITION BY column ORDER BY column)

  - PARTITION BY: Divides rows into groups (like GROUP BY, but for windows).
  - ORDER BY: Orders rows within each window.

- **Common Functions:**

  - ROW_NUMBER(): Assigns a unique sequential number to each row within a partition.
  - RANK() and DENSE_RANK(): For ranking with handling of ties.
  - LAG() and LEAD(): Access data from a previous or following row.

- **Real-World Example:** Calculate a running total of sales per month, or rank employees by salary within each department . sql SELECT employee_id, department, salary, RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dept_salary_rank FROM employees;

E. Common Table Expressions (CTEs) and Recursive Queries

- **CTE:** A temporary named result set that exists only for the duration of a query. It makes complex queries more readable and modular . sql WITH HighEarners AS (SELECT * FROM Employee WHERE salary > 80000) SELECT * FROM HighEarners WHERE dept_id = 10;
- **Recursive CTE:**
  A CTE that references itself. Essential for hierarchical data .
  - **Use Case:** Traversing an organizational chart (find all employees under a manager), or a bill of materials.
  - **Real-World Analogy:** Finding all your ancestors in a family tree – you start with yourself, then find your parents, then their parents, and so on.

F. Stored Procedures and Functions

- **Stored Procedure:**
  A set of pre-compiled SQL statements stored on the server. It can accept parameters and perform complex operations .
  - **Advantages:** Reusability, performance (pre-compiled), reduced network traffic, security.

- **User-Defined Function (UDF):**
  Similar to a procedure but
  must return a value
  . Can be used inside SQL expressions (e.g., in SELECT).
  - **Example:** A function CalculateDiscount(price) that returns a discounted price.

G. Triggers

- A trigger is a stored program that automatically executes (fires) in response to specific events (INSERT, UPDATE, DELETE) on a table .
- **Use Cases:**

  - **Audit Logging:** Automatically log who changed what and when.
  - **Enforcing Complex Business Rules:** Rules too complex for simple CHECK constraints.
  - **Maintaining Derived Data:** Automatically update a last_updated timestamp column.

H. Transaction Control (COMMIT and ROLLBACK)

- **Transaction:** A logical unit of work (one or more SQL statements) that must be executed atomically (all or nothing).
- **COMMIT:** Permanently saves all changes made in the current transaction .
- **ROLLBACK:** Undoes all changes made since the last COMMIT, restoring the database to its previous consistent state .
- **SAVEPOINT:** Creates a point within a transaction to which you can later roll back.
- **Real-World Example:** Bank transfer: Debit from A (UPDATE), Credit to B (UPDATE). If either fails, ROLLBACK. If both succeed, COMMIT. This ensures money isn't lost or created.

Unit 4: Relational Database Design (5 Hrs)

This unit is about the *theory* of good design. A well-designed database prevents data anomalies and ensures long-term data integrity.

1. Functional Dependencies (FDs)

- **Definition:** A constraint between two sets of attributes in a relation. X → Y (read as "X functionally determines Y") means that each value of X is associated with **exactly one** value of Y .
- **Real-World Analogy:** In a company, Employee_ID → Employee_Name. If you know the Employee ID, you can uniquely determine their name. Another person with that same ID *cannot* have a different name.
- **Trivial FD:** X → Y where Y is a subset of X (e.g., {A, B} → {A}). Always true.
- **Armstrong's Axioms:** Inference rules for FDs (Reflexivity, Augmentation, Transitivity). Used to find the **closure** of a set of FDs (all FDs implied by them).
- **Closure of Attribute Set (X^+):** The set of all attributes that are functionally determined by X. Used to find candidate keys.

2. Normal Forms

Normalization is a step-by-step process to decompose larger tables into smaller, well-structured ones to eliminate redundancy and update anomalies . The goal is to ensure each table describes **one** clear concept.

A. First Normal Form (1NF)

- **Rule:** The table must have a **primary key**, and all attributes must be **atomic** (no repeating groups or multi-valued attributes).
- **Violation Example:** A Student table with a Courses column containing "C01, C02, C03".
- **Fix:** Create a separate Enrollment table linking Student_ID and Course_ID.

B. Second Normal Form (2NF)

- **Rule:** Must be in 1NF, and remove **partial dependencies**. Every non-key attribute must be fully functionally dependent on the **entire primary key** (not just part of it) .
- **Applicability:** Only relevant for tables with composite primary keys.
- **Violation Example:** Enrollment(Student_ID, Course_ID, Student_Name, Grade). Student_Name depends only on Student_ID (part of the key), not the full key.
- **Fix:** Split into Student(Student_ID, Student_Name) and Enrollment(Student_ID, Course_ID, Grade).

C. Third Normal Form (3NF)

- **Rule:** Must be in 2NF, and remove **transitive dependencies**. No non-key attribute should depend on another non-key attribute .
- **Violation Example:** Employee(Emp_ID, Emp_Name, Dept_ID, Dept_Name). Dept_Name depends on Dept_ID, which is a non-key attribute.
- **Fix:** Split into Employee(Emp_ID, Emp_Name, Dept_ID) and Department(Dept_ID, Dept_Name).
- **3NF is usually sufficient** for most practical applications. It allows NULLs in foreign keys.

D. Boyce-Codd Normal Form (BCNF)

- **Definition:** A stricter version of 3NF. A table is in BCNF if for **every** non-trivial functional dependency X → Y, X must be a **superkey** .
- **The "Blind Spot" in 3NF:** 3NF only checks dependencies on **non-key** attributes. BCNF also checks dependencies where the left-hand side is part of a candidate key .
- **When 3NF is not BCNF:** This occurs when there are **overlapping candidate keys** (composite keys that share attributes).
- **Example of Violation:**

  Student_Subject_Teacher(Student_ID, Subject, Teacher)
  . Assume:
  1. Each Subject has only one Teacher (Subject → Teacher).
  2. A Student can take a Subject from only one Teacher, but a Teacher teaches many Subjects (Student_ID, Subject → Teacher). The candidate keys are (Student_ID, Subject) and possibly (Student_ID, Teacher).

  - The FD Subject → Teacher has a left-hand side (Subject) that is **not a superkey**. This violates BCNF, even though the table might be in 3NF.

- **Fix:** Decompose into Subject_Teacher(Subject, Teacher) and Student_Subject(Student_ID, Subject).

E. Comparison: 3NF vs. BCNF

Feature	3NF	BCNF
Rule	No transitive dependencies on non-key attributes.	Every determinant (LHS of FD) must be a superkey.
Tolerance	Allows some redundancy if it preserves dependencies.	Very strict; eliminates all redundancy based on FDs.
Dependency Preservation	Can always find a decomposition that preserves all FDs.	Some BCNF decompositions may lose dependency preservation .
Trade-off	Good balance for most systems.	May require extra joins, but has no update anomalies.

Guideline from Database Experts: "All attributes must provide a fact about the key, the whole key, and nothing but the key" . BCNF is the mathematical formalization of "nothing but the key."

3. Atomic Domains

- **Definition:** A domain is atomic if elements of the domain are considered **indivisible units** . This is the foundation of 1NF.
- **Example:** A phone_number column storing multiple numbers like "9801234567, 9812345678" is **not atomic**.
- **Why Important:** Atomic values are essential for reliable querying (e.g., finding all students with a specific phone number becomes impossible with multi-valued columns).

4. Decomposition using Functional Dependencies

Decomposition is the process of splitting a relation into multiple relations to achieve a higher normal form . A good decomposition must have two key properties:

A. Lossless-Join Decomposition (Non-Additive)

- **Definition:** When you join the decomposed tables back together, you must be able to reconstruct the **exact original relation** without generating spurious (extra, incorrect) tuples .
- **Guarantee:**
  A binary decomposition of R into R1 and R2 is lossless if:
  - R1 ∩ R2 → R1 (the common attributes functionally determine R1), OR
  - R1 ∩ R2 → R2 (the common attributes functionally determine R2).

- **In simple terms:** The common attribute(s) must be a **key** for one of the resulting tables.
- **Real-World Analogy:** You break a Lego castle into sections (decompose). If your decomposition is lossless, you can perfectly reassemble the original castle. If it's lossy, you end up with extra pieces that don't fit or a misshapen castle.

B. Dependency Preservation

- **Definition:** The set of Functional Dependencies (FDs) on the original schema should be enforceable by checking the FDs on the decomposed schemas individually, without needing to perform a join .
- **Why Important:** If a decomposition is not dependency-preserving, you may have to perform expensive joins just to check if a simple database update violates a constraint.
- **Real-World Analogy:** Each new table (department) can enforce its own rules (dependencies) independently. You shouldn't have to call a meeting of all departments (perform a join) to check a simple rule about one department.

C. The Trade-off (BCNF vs. 3NF)

- **BCNF** guarantees lossless joins but **may not preserve all dependencies** .
- **3NF** guarantees **both** lossless joins and dependency preservation, but may have some residual redundancy.
- **Practical Decision:** If a BCNF decomposition preserves all dependencies, use BCNF. If it doesn't, you may choose to stop at 3NF to maintain dependency preservation, especially in update-intensive applications .

Exam Tips for Units 3 & 4

- **Unit 3 (SQL):** Practice writing queries! For window functions, remember the OVER() clause. For CTEs, remember the WITH keyword. Be able to explain *why* you'd use an advanced feature over a basic one.
- **Unit 4 (Normalization):**
    - **Memorize the definitions** of 1NF, 2NF, 3NF, and BCNF with examples.
    - **Understand the difference:** 2NF removes *partial* dependencies (part of key), 3NF removes *transitive* dependencies (non-key → non-key), BCNF removes dependencies where the left-hand side is not a superkey (even if it's part of a key) .
    - **Practice decomposition:** Given a relation and FDs, find the candidate keys, check normal form, and decompose step-by-step into BCNF .
    - **Always justify** why a decomposition is lossless (common attribute is key in one table) and whether it preserves dependencies.

Unit 5: Application Design & Development (5 Hrs)

This unit bridges the gap between the database and the end-user. It focuses on how to build safe, efficient, and user-friendly database applications.

1. User Interface & Tools

- **Goal of UI in Databases:** To provide an intuitive and efficient way for users to interact with the underlying data without needing to know complex SQL .
- **Types of Database Interfaces:**
    - **Menu-Based Interfaces:** Common in web applications for non-technical users (e.g., online shopping site).
    - **Forms-Based Interfaces:** Designed for data entry and retrieval (e.g., a student registration form).
    - **Graphical User Interface (GUI):** Uses visual metaphors like diagrams, sliders, and drag-and-drop to build queries (Visual Query Systems) .
    - **Natural Language Interfaces:** Allow users to ask questions in plain English (e.g., "Show me all sales from last month").
    - **Mobile & Touch Interfaces:** Designed for small screens and touch interaction.
- **Key Consideration:** The interface must be designed with the user's mental model in mind, hiding the complexity of the backend database . A poorly designed UI can lead to user errors and low productivity, even if the database is perfectly normalized.

2. Web Fundamentals

- **Client-Server Architecture:**
  The web operates on this model.
  - **Client (Browser):** (e.g., Chrome, Firefox) Responsible for the presentation layer (HTML, CSS, JavaScript). It sends requests and displays the results .
  - **Server (Web/Application Server):** (e.g., Apache Tomcat, Nginx) Responsible for the business logic. It receives requests, processes them (often by talking to a database), and sends back a response .
  - **Real-World Analogy:** You (the client) walk into a restaurant and give your order to the waiter. The waiter takes the order to the kitchen (the server). The kitchen prepares the meal and gives it back to the waiter, who then serves it to you.

- **HTTP Protocol:**
  The language of communication.
  - **Request:** The client asks for a resource (a page, an image). It includes a URL, method (GET, POST), headers, and possibly data.
  - **Response:** The server replies with a status code (e.g., 200 OK, 404 Not Found), headers, and the requested content.

- **Statelessness:** HTTP is stateless. Each request is independent. To remember a user between requests (e.g., keeping them logged in), we need **session management** (cookies, URL rewriting).

3. Servlets and JSP

These are Java Enterprise (JEE) technologies for building server-side logic .

A. Servlets (The Controller)

- **Definition:** A Java program that runs on a web server (inside a **servlet container** like Apache Tomcat's Catalina engine) and dynamically processes requests and generates responses .
- **Lifecycle (Managed by the Container):**

  1. **Loading and Instantiation:** Container loads the servlet class and creates an instance.
  2. **Initialization (init()):** Called once when the servlet is first loaded. Used for setting up database connections or other resources .
  3. **Request Handling (service()):** For each client request, the container calls the service() method. This method typically calls doGet() or doPost() based on the HTTP method .
  4. **Destruction (destroy()):** Called once when the servlet is being taken out of service. Used for cleanup.

- **Key Objects:** HttpServletRequest (to read data from the client, like form parameters) and HttpServletResponse (to send a response back to the client) .

B. JSP (JavaServer Pages) (The View)

- **Definition:** A technology that helps create dynamically generated web pages based on HTML, but with special tags for Java code. They are eventually compiled into Servlets by an engine like Jasper .
- **Purpose:** To separate **presentation** (HTML/CSS) from **business logic** (Java code). This makes web applications easier to manage.

C. MVC Architecture (Model-View-Controller)

This is the standard design pattern for web applications.

- **Model:** The data and business logic (e.g., JavaBeans/POJOs, Database classes).
- **View:** What the user sees (e.g., JSP pages).
- **Controller:** The traffic cop that handles requests, interacts with the Model, and selects the View to display (e.g., a Servlet) .
- **Advantages:** Separation of concerns, easier maintenance, and code reusability.

4. Authorization in SQL

Authorization is about controlling what actions a user can perform on the database objects. This is managed through a privilege system.

- **Roles:**
  A named collection of privileges. It's easier to grant privileges to a role and then assign users to that role, rather than granting privileges to each user individually .
  - **Real-World Analogy:** Instead of giving every new employee a set of keys individually, you give them a role (e.g., "Sales Team") which already has the necessary keys.

- **Creating and Managing Roles:**
  - CREATE ROLE role_name; (e.g., CREATE ROLE clerk;)
  - GRANT role_name TO user_name; (e.g., GRANT clerk TO john;) This allows john to use the privileges of the clerk role .

- **Privileges:**
  Specific actions allowed on database objects.
  - **Types:** SELECT, INSERT, UPDATE, DELETE, REFERENCES, ALL PRIVILEGES.

- **Granting Privileges:**
  - GRANT SELECT, INSERT ON student TO clerk;
  - This gives the clerk role the ability to read and insert data into the student table.

- **Revoking Privileges:**
  - REVOKE INSERT ON student FROM clerk;
  - This takes away the insert permission from the clerk role.

- **Hierarchy of Roles:** Roles can be granted to other roles, creating a hierarchy where privileges flow downward . For example, GRANT engineer TO manager; gives the manager role all privileges of the engineer role.

5. Application Security

This is about protecting the database from malicious attacks or accidental misuse.

A. SQL Injection

- **Definition:** A code injection technique where an attacker inserts malicious SQL code into a query parameter (e.g., a login form or search box) to manipulate the database .
- **How it works:** The application dynamically builds SQL queries by concatenating strings from user input without proper validation.
- **Example of a Vulnerable Login Query:** sql -- User inputs username: 'admin' and password: 'anything' OR '1'='1' "SELECT * FROM users WHERE username = '" + username + "' AND password = '" + password + "'"; The resulting query becomes: sql SELECT * FROM users WHERE username = 'admin' AND password = 'anything' OR '1'='1'; Since '1'='1' is always true, the attacker can log in as admin without knowing the password .
- **Consequences:** Authentication bypass, data theft (downloading entire tables), data loss (deleting tables), and even gaining control of the database server .

B. Mitigation Strategies (How to Prevent It)

- **1. Parameterized Queries (Prepared Statements):** The most effective defense. This separates the SQL code from the data. The query structure is defined first, and user input is passed in as parameters, never as part of the SQL string . java // Secure example in Java using PreparedStatement String sql = "SELECT * FROM users WHERE username = ? AND password = ?"; PreparedStatement pstmt = connection.prepareStatement(sql); pstmt.setString(1, username); // username is treated as data, not code pstmt.setString(2, password); ResultSet rs = pstmt.executeQuery();
- **2. Principle of Least Privilege:** Database accounts used by the application should have the minimum permissions necessary. If the app only needs to read data, don't give it INSERT, UPDATE, or DELETE privileges . This limits the damage of a successful attack.
- **3. Input Validation:** Validate all user input against an **allowlist** of acceptable values (e.g., if a field should be a number, reject anything that isn't a number). Do not rely solely on denylists .
- **4. Stored Procedures:** While not always foolproof, using stored procedures can add a layer of abstraction and security if implemented correctly (and not using dynamic SQL inside them).
- **5. Escape Special Characters:** If you cannot use parameterized queries (rare), ensure all special characters (like quotes) in user input are escaped so they are not interpreted as SQL syntax .

Unit 6: Transaction Processing, Concurrency Control & Recovery (7 Hrs)

This is the most critical unit for understanding how databases maintain correctness and reliability in a multi-user environment.

1. Transaction Concept

- **Definition:** A transaction is a **logical unit of work** comprising one or more SQL operations (reads and writes). It is the fundamental unit of recovery and concurrency .
- **Real-World Example:**
  A bank transfer of $100 from Account A to Account B. The transaction consists of:
  1. Read balance of A.
  2. Debit $100 from A (Write).
  3. Read balance of B.
  4. Credit $100 to B (Write).
  5. Commit.

2. ACID Properties

To ensure correctness, every transaction must be **ACID** compliant.

- **Atomicity:** The "all-or-nothing" property. The transaction must execute completely or not at all. If the system crashes after step 2 in the example, the money must not be deducted from A without being added to B. The database must roll back the partial changes .
- **Consistency:** The transaction must take the database from one **valid state** to another, preserving all integrity constraints (keys, check constraints). If a transaction violates a rule, it is aborted .
- **Isolation:** Transactions must appear to execute in **isolation**, as if they were running one after another (serially), even though they may be executing concurrently. The intermediate state of a transaction should be invisible to other transactions .
- **Durability:** Once a transaction commits, its changes are **permanent** and must survive any subsequent system failure (crash, power outage) .

3. Concurrent Execution & Problems

Running transactions concurrently is essential for performance, but it can lead to anomalies if not controlled.

- **Lost Update:** Two transactions read the same value (e.g., X=100) and both try to update it. T1: X = X + 50. T2: X = X + 20. The final result could be 120 (T2's update lost) or 150 (T1's update lost), instead of the correct 170.
- **Dirty Read (Temporary Update):** A transaction (T1) reads data that has been updated by another transaction (T2) which has **not yet committed**. If T2 later aborts (rolls back), T1 has read data that never existed (dirty data) .
- **Unrepeatable Read:** A transaction (T1) reads the same data twice. Between the two reads, another transaction (T2) updates that data and commits. T1 sees two different values for the same read, which is inconsistent.

- **Phantom Read:** A transaction (T1) runs a query (e.g., SELECT * FROM student WHERE age > 20) twice. Between the two queries, another transaction (T2) inserts a new row that satisfies the condition (age=21) and commits. T1 sees a "phantom" row in its second read that wasn't there the first time.

4. Serializability & Recoverability

These are the criteria for a correct concurrent schedule.

- **Schedule (History):** A sequence of operations (reads/writes) from a set of transactions, showing the order in which they are executed .
- **Serial Schedule:** A schedule where transactions are executed one after another, with no interleaving. This is guaranteed to be correct but offers poor performance.
- **Serializability:** The property of a **concurrent** schedule that its effect is equivalent to that of **some** serial schedule. This is the standard for correctness in concurrency control .
- **Conflict Serializability:**
  A schedule is conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.
  - **Conflicting Operations:** Two operations conflict if they belong to different transactions, operate on the same data item, and at least one of them is a write.
  - **Serializability Graph:** A directed graph where nodes are transactions. An edge T1 → T2 exists if T1 has a conflicting operation that occurs before a conflicting operation of T2. A schedule is conflict serializable **if and only if its serializability graph is acyclic** (has no cycles) .

- **Recoverability:** Ensures that transactions are committed only after all transactions whose changes they read have been committed. This prevents dirty reads and ensures we never have to roll back a committed transaction.

5. Concurrency Control Protocols

Mechanisms to ensure serializability.

A. Lock-Based Protocols

Transactions acquire locks on data items before accessing them.

- **Shared Lock (S-Lock):** For reading only. Multiple transactions can hold a shared lock on the same item simultaneously.
- **Exclusive Lock (X-Lock):** For writing. Only one transaction can hold an exclusive lock on an item.

B. Two-Phase Locking (2PL)

A protocol that guarantees conflict serializability.

- **Phase 1: Growing Phase:** A transaction may acquire locks but cannot release any.
- **Phase 2: Shrinking Phase:** A transaction may release locks but cannot acquire any new ones.
- **Lock Point:** The moment when the transaction has acquired all its locks. This defines the serialization order.
- **Strict 2PL:** A common variant where a transaction holds all its exclusive locks until after it commits or aborts. This avoids cascading rollbacks and is widely used.

C. Timestamp-Based Protocols

Assigns a unique timestamp to each transaction. The system ensures that conflicting operations are executed in timestamp order.

- If a younger transaction tries to access data in a way that violates the timestamp order, it is aborted and restarted with a new timestamp.

D. Multiversion Concurrency Control (MVCC)

- **Concept:** Instead of locking a data item for a reader, the database keeps multiple versions of the data item. A writer creates a new version; a reader reads the version that was current when its transaction started.
- **Advantage:** Readers never block writers, and writers never block readers, leading to high concurrency . This is used by modern databases like PostgreSQL and Oracle.

6. Deadlock Handling

A deadlock is a situation where two or more transactions are each waiting for a lock held by the other, and neither can proceed .

- **Necessary Conditions for Deadlock (Coffman Conditions) :**

  1. **Mutual Exclusion:** Resources cannot be shared.
  2. **Hold and Wait:** A transaction holds at least one resource and is waiting for additional resources held by others.
  3. **No Preemption:** A resource cannot be forcibly taken away from a transaction.
  4. **Circular Wait:** A cycle of transactions exists, where each is waiting for the next in the cycle.

- **Deadlock Prevention:**
  Ensures that at least one of the four conditions never occurs.
  - **Prevent Circular Wait:** Order all data items and require transactions to acquire locks in a predefined order (e.g., ascending order). This is the most common prevention method .

- **Deadlock Detection and Recovery:**

  - **Detection:** The system periodically checks for a cycle in the **wait-for graph** (a graph showing which transaction is waiting for which other transaction). A cycle indicates a deadlock .

- **Recovery:** The system chooses a **victim** transaction (often the one with the least cost, like the youngest or one holding the fewest locks) and **aborts** it, releasing its locks and allowing others to proceed. This is called the **optimistic approach** .

7. Recovery Model of Various Failures

Recovery mechanisms ensure **Atomicity** and **Durability** in the face of failures .

A. Classification of Failures

- **Transaction Failure:**
  - **Logical Error:** Transaction cannot complete due to an internal error (e.g., division by zero, integrity constraint violation).
  - **System Error:** The database system itself has to abort the transaction (e.g., due to deadlock).

- **System Crash (Soft Failure):** A power failure or operating system crash that causes the database system to stop abruptly. The contents of the main memory (buffer pool) are lost, but the disk (secondary storage) is intact .
- **Disk Failure (Hard Failure):** A physical failure of the disk head or controller. This results in loss of the database on the disk .

B. Recovery Strategies: Log-Based Recovery

The most widely used technique is to maintain a **log file** (a sequential record of all database modifications). This log must be stored on stable storage (a separate, reliable disk) and is written **ahead** of the actual data changes (Write-Ahead Logging - WAL).

- **Key Recovery Concepts :**
  - **STEAL / NO-STEAL:** Can the buffer manager write an uncommitted transaction's updates to disk before the transaction commits? If **STEAL** is allowed (modern DBMSs do this for performance), we may need **UNDO** logging.
  - **FORCE / NO-FORCE:** Must all updates of a committed transaction be forced to disk immediately at commit time? If **NO-FORCE** is used (for performance, to avoid waiting for disk I/O), we may need **REDO** logging.
  - Modern DBMSs like those using the **ARIES** algorithm use **STEAL / NO-FORCE** for maximum performance. This requires both UNDO and REDO logging .

- **Types of Log Records :**
  - **UNDO:** Information needed to reverse the changes of an uncommitted transaction (e.g., the old value of a data item). Used for **Atomicity**.
  - **REDO:** Information needed to reapply the changes of a committed transaction whose updates may not yet be on disk after a crash (e.g., the new value). Used for **Durability**.

- **Checkpoint:** A record in the log that signifies that all transactions before this point have been committed and their changes have been written to disk. It marks a point up to which we know the data is safe, limiting how far back we need to scan the log during recovery .

C. Recovery Process After a System Crash

1. **Analysis Phase:** Scan the log forward from the last checkpoint to identify all transactions that were active at the time of the crash.
2. **Redo Phase (Roll Forward):** Scan the log from the last checkpoint and **REDO** all actions (for both committed and uncommitted transactions) to ensure the database state reflects all changes up to the crash .
3. **Undo Phase (Roll Back):** Scan the log backward and **UNDO** all actions of the transactions that were active at the time of the crash (those that did not commit), restoring the database to a consistent state .

- **Recovery After a Media (Disk) Failure:**

  1. **Restore:** The DBA restores the database from the most recent full backup .
  2. **Rollforward:** The system then applies all committed transactions from the archived transaction logs (the log files backed up since the last backup) to bring the database to the most current state possible.

Unit 7: Introduction to Object Oriented Database (3 Hrs)

This unit introduces a paradigm shift from the relational model to one that aligns with object-oriented programming languages.

1. General Concepts of Object-Oriented Database (OODB)

- **Definition:** An Object-Oriented Database (OODB) is a system that stores data as **objects**, similar to object-oriented programming (OOP), rather than in tables with rows and columns .
- **Core Idea:**
  It integrates database capabilities with OOP features. An object contains both
  data
  (attributes) and
  behavior
  (methods/operations) .
  - **Real-World Analogy:** Think of a "Car" object. Its data includes color, model, speed. Its behavior includes accelerate(), brake(), changeGear(). In a relational DB, you'd store color and model in a table, but the behavior would be in application code. In an OODB, everything is bundled together.

- **Goal:** To bridge the "**impedance mismatch**" between the object-oriented application model and the relational database model, providing a seamless integration for developers .

2. Core Object-Oriented Concepts in Database Management

OODBs are built upon the fundamental principles of OOP .

- **Objects and Classes:**
  - **Class:** A blueprint or template that defines the structure (attributes) and behavior (methods) for a set of objects .
  - **Object:** An **instance** of a class. For example, the Book class is the blueprint, while the physical copy of "Database Systems Concepts" in your hand is an object (instance) of that class .

- **Encapsulation:**
  - **Definition:** Bundling data (attributes) and the methods that operate on that data into a single unit (the object), and restricting direct access to some of the object's internal components .
  - **Real-World Example:** Your smartphone. You interact with its behavior through a well-defined interface (the touchscreen, buttons). You don't directly manipulate the voltage in the battery or the raw data in the memory. The internal details are **encapsulated** .
  - **Benefit:** Data security and modularity.

- **Inheritance:**
  - **Definition:** A mechanism for creating a new class (child class) from an existing class (parent class). The child class inherits the attributes and methods of the parent and can add its own unique features .
  - **Real-World Example:** A parent class Vehicle has attributes speed and color. A child class Car inherits these and adds its own attribute, numberOfDoors. Another child class Bike inherits from Vehicle and adds hasBasket .
  - **Benefit:** Code reusability and a hierarchical organization of data.

- **Polymorphism:**
  - **Definition:** The ability of an object to take on many forms. More specifically, it allows the same method name to have different implementations in different classes .
  - **Real-World Example:** A method named draw() can be called on different shape objects. A Circle object will have a draw() method that draws a circle, while a Square object will have a draw() method that draws a square. The same interface yields different behaviors .
  - **Benefit:** Flexibility and extensibility.

3. Key Features of OODBs

- **Object Identity (OID):** Every object in the database has a unique, system-generated identifier that is independent of its attribute values. This is a direct implementation of the object-oriented concept of identity, where an object remains the same entity even if its attributes change .
- **Complex Objects:** OODBs can directly store complex data structures built from simpler ones using constructors like sets, lists, and tuples. This allows for modeling intricate real-world relationships naturally .
- **Type Hierarchies and Inheritance:** As discussed, this is a core feature, allowing for structured and reusable data definitions .
- **Persistence:** Objects are stored persistently, meaning they exist beyond the lifetime of the program that created them. This is **orthogonal to type**—any object, regardless of its type, can be made persistent .
- **Query Capability:** While navigation via pointers is primary, OODBMS also provide an **ad-hoc query facility** (like SQL but for objects, e.g., OQL) to find objects declaratively .

4. Object-Oriented vs. Object-Relational Databases

This distinction is crucial for the exam. They are often collectively called "object databases" .

Feature	Object-Oriented Database (OODB)	Object-Relational Database (ORDB)
Foundation	Built from scratch on the object-oriented paradigm .	Built on top of an existing relational database system by adding object-oriented features .
Data Model	Directly supports objects, classes, and inheritance as primary constructs .	Extends the relational model with object-oriented concepts like user-defined types (UDTs), typed tables, and references (similar to OIDs) .
Query Language	Object Query Language (OQL), which is designed to navigate and retrieve objects.	Extended SQL (e.g., SQL:1999 and later) with constructs for UDTs, methods, and nested tables.
Integration	Tight, seamless integration with OOP languages like C++, Java, or Smalltalk. The database feels like a native part of the language .	Acts as a bridge. The database is still relational at its core but can store and manipulate objects, often using a framework for mapping .
Use Case Analogy	A custom-built boutique hotel where every aspect is designed around the OOP concept.	A classic hotel that has been renovated and extended with modern, object-oriented wings and amenities.

Unit 8: Introduction to Distributed Database (3 Hrs)

This unit deals with a database physically stored across multiple computers but appearing as a single logical unit to the user.

1. General Concepts of Distributed Database

- **Definition:** A **distributed database** is a collection of multiple, logically interrelated databases distributed over a computer network .
- **Distributed Database Management System (DDBMS):** The software that manages the distributed database, making the distribution **transparent** to the users .
- **Key Principle:** Users should feel they are interacting with a single, centralized database, unaware of the underlying fragmentation, replication, and distribution of data across different sites.

2. Distributed Transaction Management

A distributed transaction accesses and updates data stored on multiple different computers (nodes) in the network.

- **Challenges:**
  - **Atomicity:** Ensuring that either all the sub-transactions on all participating nodes commit, or they all abort. A failure on one node should cause the entire global transaction to roll back .
  - **Two-Phase Commit (2PC):**
    A classic protocol to achieve atomicity for distributed transactions.
    - **Phase 1 (Prepare):** A coordinator node asks all participants if they can commit their part of the transaction.
    - **Phase 2 (Commit/Abort):** If all participants vote "Yes," the coordinator tells them all to commit. If any votes "No," the coordinator tells them all to roll back .

- **Recovery:** More complex, as failures can be of individual nodes, communication links, or the network itself. Protocols must handle these to maintain consistency .

3. Concurrency Control in Distributed Databases

Concurrency control ensures isolation in a distributed environment. Standard locking protocols are extended to work across multiple sites.

- **Distributed Locking:** Locks are managed at each local site where a data item resides. A transaction may need to obtain locks from multiple lock managers.
- **Deadlock Handling:**
  More complex, as the wait-for graph is distributed.
  - **Centralized Approach:** One node is responsible for collecting lock information and detecting cycles.
  - **Hierarchical Approach:** Deadlock detection is distributed across a hierarchy of nodes .

- **Timestamping:** Each transaction is assigned a globally unique timestamp. The order of transactions is determined by these timestamps to ensure serializability, even across sites.

4. Distributed Query Processing

Processing a query in a distributed database is more complex than in a centralized one. The goal is to minimize the cost of data transfer across the network, which is often the dominant cost.

- **Phases of Distributed Query Processing :**

  1. **Query Decomposition:** The SQL query is broken down into smaller, algebraic operations on the global schema (similar to centralized query processing).
  2. **Data Localization:** The query is modified to specify which fragments (e.g., "employees in the Kathmandu branch") are involved. This generates a query on fragments.
  3. **Global Query Optimization:** The system chooses the best strategy for executing the fragment query. This involves deciding the order of operations, where to perform joins (e.g., ship both tables to a single site or do a semi-join), and how to use replicated data to reduce network traffic.
  4. **Local Query Optimization:** The local DBMS at each participating site further optimizes its part of the query.

Unit 9: Database System Architecture (3 Hrs)

This unit covers the different ways to structure the components of a database system, from a single machine to a distributed network.

1. Client/Server Architecture

- **Definition:** A model where tasks or workloads are partitioned between **providers of a resource or service (servers)** and **service requesters (clients)** .
- **Real-World Analogy:** You (the client) go to a restaurant and order a meal from the waiter. The waiter takes your order to the kitchen (the server). The kitchen prepares the meal and gives it back to the waiter, who then serves it to you.
- **Database Context:** The client (often a personal computer running an application) manages the user interface and application logic, while the database server (a powerful machine) manages the data, transaction processing, and security .

2. Tier Architecture

This refers to the physical separation of the presentation, application logic, and data management layers .

Architecture	Structure	Description	Real-World Analogy	Pros & Cons

1-Tier	All components (UI, business logic, data) are on a single machine.	The database and the application that accesses it reside on the same system.	A personal cook who shops, preps, cooks, and serves a meal entirely in their own kitchen.	**Pros:** Simple, fast (no network latency). **Cons:** No separation of concerns, not scalable, poor security. Used for local applications (e.g., MS Access).
2-Tier	Client (Tier 1) directly communicates with the Database Server (Tier 2).	The client handles the UI and business logic. It sends SQL queries directly to the database server for processing. Results are sent back.	A customer (client) going to a restaurant counter and directly giving their order to the cook (server).	**Pros:** Simple to implement, good for small to medium applications. **Cons:** The client becomes "fat" (handles too much logic), scalability limited, security risks as DB credentials may be on the client. .
3-Tier	Client (Tier 1) <--> Application Server (Tier 2) <--> Database Server (Tier 3).	This is the standard for modern web applications. The client (web browser) only handles UI. It sends requests to the **Application Server** (e.g., Tomcat, JBoss), which contains the business logic. The application server then interacts with the **Database Server** .	A customer (client) ordering from a waiter (application server). The waiter checks the order, then relays it to the chef (database server). The chef prepares the meal and gives it to the waiter, who serves the customer.	**Pros:** "Thin" clients, improved security (client never touches DB), scalability (you can add more application servers), load balancing, and better maintainability .
N-Tier (Multi-Tier)	Further separation of the application server into multiple, specialized tiers.	For highly complex, large-scale enterprise systems. You might have separate tiers for web servers, application servers, integration servers, and legacy system adapters .	A large hotel chain with a central reservation system, regional booking offices, and local hotel management systems, all communicating in a structured way.	**Pros:** Maximum flexibility, scalability, and reusability. **Cons:** High complexity, cost, and overhead in development and management.

3. Distributed Systems Architecture

This refers to the architecture of the DDBMS itself, describing how the database management is distributed across multiple nodes .

- **Architectural Models for Distributed DBMS:**
  - **Client-Server Distributed Architecture:** A natural extension of the 2/3-tier models, where clients send requests to multiple distributed database servers.
  - **Peer-to-Peer Distributed Architecture:** Each node in the system acts as both a client and a server, sharing its own data and processing requests from others. No central coordination is implied.
  - **Multidatabase System (MDBS):** A layer of software on top of existing, autonomous, and possibly heterogeneous local databases. It provides a single, integrated view to users, allowing them to query across multiple different DBMSs (e.g., an Oracle DB and a MySQL DB) as if they were one. This is also known as a **federated database system** .

Exam Tips for Units 7, 8 & 9

- **Unit 7:** Be crystal clear on the difference between **OODB and ORDB**. Know the **OOP concepts** (inheritance, polymorphism, encapsulation) with your own simple example (like Vehicle -> Car, Bike). The OODBMS Manifesto list is a goldmine for long-answer questions on features.
- **Unit 8:** Focus on the **challenges** introduced by distribution. For transactions, the **two-phase commit (2PC)** protocol is a must-know. For queries, remember the four-step process. The **deadlock** discussion is also a key topic.
- **Unit 9:** Be able to draw and explain the **3-Tier Architecture** with a clear example (like an online shopping site). Compare and contrast it with the simpler 2-Tier model, highlighting the advantages in security, scalability, and maintainability .

Unit 10: Advanced Transaction Processing (7 Hrs)

This unit moves beyond the traditional ACID transaction model to address the requirements of modern, complex, and distributed applications. It introduces transaction models and management techniques for workflows, e-commerce, real-time systems, long-duration activities, and multidatabase environments.

1. Transactional Workflows

A. General Concepts

- **Definition:** A **transactional workflow** is a collection of tasks (activities or steps) organized to accomplish a business process, where the overall execution must guarantee transactional properties (like atomicity, consistency, isolation, and durability) .
- **Core Idea:** It extends the traditional transaction model to support **long-running, multi-step business processes** that span multiple applications, databases, and organizations. Each task may be a transaction in itself .
- **Real-World Analogy:** A loan application process. It involves multiple steps: application submission, credit check, document verification, manager approval, and fund disbursement. Each step is a task, and the entire process must be reliable—if approval fails after credit check, the whole application might need to be rolled back or compensated.

B. Key Characteristics

- **Multi-step Process:** Composed of multiple tasks (activities) executed in a specific order (sequence, parallel, conditional) .
- **Heterogeneous:** Tasks may execute on different systems, databases, or even across different organizations .
- **Long Duration:** Workflows can run for hours, days, or even weeks, making traditional locking impractical .
- **Relaxed Isolation:** Intermediate results may be visible to other workflows (unlike ACID transactions where isolation is strict) .
- **Compensation:** For each task, there is often a compensating task (a "rollback" activity) to undo its effects if the workflow fails later.

C. Transactional Properties in Workflows

- **Atomicity:** Ensures that either the entire workflow completes successfully, or if it fails, all completed tasks are compensated (rolled back via compensating transactions) .
- **Consistency:** The workflow moves the system from one consistent state to another. This may involve checking business rules after each task or at the end .
- **Isolation:** Typically **relaxed**. Intermediate results are often visible (e.g., a customer can see their "loan application submitted" status before final approval).
- **Durability:** Once the workflow commits (successfully completes), all its effects are permanent.

D. Real-World Example: Vinted's Payment Workflow

- **Company:** Vinted, Europe's leading online marketplace for second-hand clothing .
- **Challenge:** Coordinating complex payment flows involving multiple payment methods, timing sensitivities, and conditional logic across their marketplace .
- **Solution:** They implemented a **workflow-based transaction system** using Temporal.
- **Key Results:**

  - They run **10 to 12 million Workflows per day** in payments alone .
  - Workflows provide **readable, top-to-bottom logic** that clarifies the entire payment process from buyer confirmation to seller payment .
  - Built-in retries, locking, and consistent execution ensure **high reliability** .

    - Parallelism is supported out of the box, speeding up flows that used to be serialized and slow .

- **Lesson:** Transactional workflows enable complex, long-running business processes to be managed reliably with clear visibility and error handling.

2. E-Commerce Transaction Processing

A. General Concepts

- **Definition:** E-commerce transaction processing refers to the management of financial transactions (orders, payments, refunds) in an online shopping environment. It requires **high availability, scalability, and reliability** .
- **Core Challenges:**

    - **Traffic Spikes:** Handling massive spikes during events like festivals, sales, or IPL matches (up to 20 times average load) .
    - **Short-lived State:** Payment transactions often maintain state only for short durations (averaging 2 minutes) while routing payments to gateways .
    - **Zero Downtime:** E-commerce platforms must maintain zero downtime for maintenance activities .
    - **Consistency:** Ensuring that inventory, orders, and payments remain consistent across the system.

B. Real-World Example: Juspay's Payment Architecture

- **Company:** Juspay, a payment orchestration platform processing over **30 million transactions daily** .
- **Business Challenge:** During IPL (Indian Premier League) cricket tournament, traffic would spike from 1x to 20x in seconds, reaching **7.6 million transactions per hour** .
- **Previous Architecture Problems:**

    - Amazon RDS (relational database) had to be vertically scaled (e.g., r5.4xlarge to r5.16xlarge) for the entire season, which was costly and complex .
    - CPU utilization during non-peak time was less than 15%, indicating inefficiency .
    - Scaling required hours of orchestration and risked downtime .

C. Solution: Memory-First Architecture with CQRS

Juspay implemented a modern architecture using Amazon ElastiCache (in-memory cache) as a write-back cache .

- **Memory-First Approach:**

    - Store active transactions temporarily in ElastiCache (in-memory) .

    - Achieve **sub-millisecond latency** for write acknowledgments .
    - ElastiCache handles up to **350,000 operations per second** with 0.44–2.45 ms latency, compared to 5–10 ms for Cassandra and 2–10 ms for RDS .

- **Write Path Optimization:**

    - Use ElastiCache as **write-ahead logs** for immediate write acknowledgment .
    - A "drainer" process writes data from cache to persistent storage (RDS) at a controlled rate based on RDS CPU utilization .

- **CQRS Pattern (Command Query Responsibility Segregation):**

    - Separate read and write models .
    - ElastiCache handles all writes (command side).
    - RDS serves as the query side for complex reads and operational use cases, with **eventual consistency** .

- **Results Achieved:**

    - Successfully process **7.6 million transactions per hour** during peak events .
    - Achieve **sub-millisecond latency** for transaction processing .
    - Reduce infrastructure costs by **80%** compared to previous solution .

D. Key Lessons for E-Commerce Transactions

- Use **in-memory caching layers** to absorb traffic spikes .
- Implement **CQRS** to optimize read and write paths independently .
- Design for **horizontal scalability** (adding more cache nodes rather than vertically scaling databases) .
- Consider **short-lived state** characteristics when choosing storage technologies .

3. Real-Time Transaction Processing

A. General Concepts

- **Definition:** Real-time transaction processing involves transactions that must be completed within **strict timing constraints (deadlines)** . A transaction that misses its deadline is considered to have failed, even if it produces correct data .
- **Key Characteristics:**

    - **Time Constraints:** Transactions have deadlines (hard, firm, or soft) .
    - **Predictability:** System must guarantee that deadlines are met under specified load conditions.
    - **Priority Scheduling:** Transactions are scheduled based on priority (often deadline-based) rather than just fairness.
    - **Temporal Consistency:** Data may have temporal validity (e.g., stock prices are valid only for a few milliseconds).

B. Real-Time vs. Traditional Transactions

Feature	Traditional Transactions	Real-Time Transactions
Success Criterion	Logical correctness only	Logical correctness + Timeliness
Scheduling	Fairness, throughput	Deadline-based priority
Concurrency Control	Locking, 2PL	Priority-based, optimistic
Data Validity	Permanent	May expire (temporal)
Examples	Bank transfer, inventory update	Stock trading, air traffic control

C. Applications

- **Stock Trading Systems:** Buy/sell orders must execute within milliseconds before prices change.
- **Network Traffic Management:** Routing decisions must be made within microseconds.
- **Industrial Control Systems:** Sensor data must be processed and acted upon within strict time windows.
- **Emergency Response Systems:** 911 call routing and dispatch must happen immediately.

D. Management Approaches

- **Priority Assignment:** Transactions are assigned priorities based on deadlines (earliest deadline first, shortest processing time first).
- **Priority Inversion Problem:** A high-priority transaction waiting for a lock held by a low-priority transaction. Solved by **priority inheritance protocols** (low-priority transaction temporarily runs at high priority to release lock faster).
- **Concurrency Control:** Real-time databases often use **optimistic concurrency control** with priority-based validation to avoid blocking.

4. Long Duration Transactions

A. General Concepts

- **Definition:** Long-duration transactions are transactions that execute over an extended period—minutes, hours, days, or even weeks—rather than milliseconds or seconds .
- **Core Challenge:** Traditional ACID transactions with **locking** are impractical because holding locks for long periods would severely impact concurrency and system performance .
- **Real-World Analogy:** A collaborative document editing session (like Google Docs) that lasts for hours. You cannot lock the entire document while one user is editing—others must be able to see and work on different parts.

B. Problems with Traditional ACID for Long Transactions

- **Locking Problem:** Holding locks for long periods reduces concurrency and can cause excessive waiting .
- **Rollback Problem:** Rolling back a long transaction after hours of work is expensive and wasteful .
- **Visibility Problem:** Intermediate results may need to be visible to other transactions (relaxed isolation).

C. Solutions and Models

- **1. Nested Transactions:**
  - A long transaction is divided into **subtransactions** (nested within a parent).
  - Subtransactions can commit or abort independently.
  - If a subtransaction aborts, the parent can retry or choose an alternative path.
  - Only when the top-level (root) transaction commits are all effects made permanent.
- **2. Sagas (Compensating Transactions):**
  - A long-running transaction is broken into a sequence of **ACID subtransactions**.
  - Each subtransaction has a corresponding **compensating transaction** that semantically undoes its effects .
  - If a step fails, compensating transactions are executed in reverse order to undo previously completed steps.
  - **Example:** A travel booking saga: (1) Book flight, (2) Book hotel, (3) Book car. If car booking fails, compensate by canceling hotel and flight (with appropriate cancellation policies).
- **3. ConTracts:**
  - A transaction model that explicitly supports **long-lived, multi-step activities** with programmed recovery procedures .
  - Steps are executed sequentially, and in case of failure, the contract can be resumed from a checkpoint or compensated.

5. Transaction Management in Multidatabase Systems

A. General Concepts

- **Definition:** A **multidatabase system (MDBS)** is a facility that allows users to access and manipulate data located in **multiple autonomous and possibly heterogeneous database management systems (DBMSs)** .
- **Core Idea:** It provides a uniform, integrated view of data spread across different databases while preserving the **autonomy** of each local database system .
- **Key Components:**
  - **Global Transactions:** Transactions executed under the control of the MDBS, accessing data from multiple local databases .
  - **Local Transactions:** Transactions executed directly under the control of each local DBMS, unaware of the global system .

- **Global Transaction Manager:** Coordinates global transactions across participating local DBMSs .

B. Characteristics of Multidatabase Systems

- **Heterogeneity:** Local DBMSs may use different data models, query languages, and transaction management schemes .
- **Autonomy:** Each local DBMS has complete control over its own data and transactions, including the ability to abort any transaction (global or local) executing at its site . The MDBS typically cannot change the design or internal structure of local DBMSs .
- **Distribution:** Data is physically distributed across multiple sites .
- **Lack of Global Control:** Local DBMSs may not be aware of each other and cannot coordinate their actions directly .

C. Challenges in Multidatabase Transaction Management

- **Autonomy vs. Global Consistency:**
  - Traditional distributed databases ensure consistency at the cost of autonomy .
  - In MDBS, **autonomy must be preserved**, meaning we cannot force local DBMSs to change their protocols or to coordinate with each other .
- **Different Local Concurrency Control:**
  - Local DBMSs may use different concurrency control protocols (locking, timestamping, optimistic) .
  - Ensuring global serializability without violating local autonomy is extremely difficult .
- **Two-Phase Commit (2PC) Limitations:**
  - 2PC requires local DBMSs to support a **prepare-to-commit** state and to follow the protocol.
  - Autonomous local DBMSs may not support this, or may abort transactions at any time for local reasons .
  - 2PC also causes **blocking** problems in multidatabase environments .
- **Global Deadlocks:**
  - Deadlocks can involve resources across multiple local DBMSs.
  - Detecting and resolving global deadlocks is complex due to autonomy and lack of global lock information .
- **Heterogeneous Transaction Models:**
  - Local DBMSs may not support the same transaction properties (e.g., one may support nested transactions, another may not) .

D. Transaction Classification in MDBS

According to Liu et al., multidatabase transactions can be classified based on their requirements :

- **Tightly-Coupled Transactions:** Require strong consistency and atomicity across multiple databases. May need special protocols.
- **Loosely-Coupled Transactions:** Allow relaxed consistency and can tolerate some inconsistencies. Use compensating transactions for recovery.

E. Approaches to Multidatabase Transaction Management

- **Relaxing ACID Properties:**
  - For scalability in large federations (thousands of members), it is necessary to relax some ACID properties .
  - Federated transactions contain **semantic knowledge** about applications and data, allowing them to be "intelligent entities" that can handle relaxed consistency .
- **Two-Phase Commit Variants:**
  - **Presumed Abort/Presumed Commit:** Optimizations to reduce 2PC overhead.
  - **Optimistic 2PC:** Reduces blocking by assuming commit will happen.
- **Compensation-Based Approaches:**
  - Use compensating transactions to handle failures, rather than requiring all-or-nothing atomicity .
  - Each global subtransaction has a compensating transaction defined.
- **Semantic Multidatabase Protocols:**
  - Exploit application semantics to allow more concurrency and relaxed consistency .
  - For example, if updates to different databases are semantically independent, they can be committed independently.

F. Tradeoffs in Protocol Selection

- Different protocols involve tradeoffs between:
  - **Transaction Autonomy:** Freedom of transactions to access data objects .
  - **System Autonomy:** Freedom of transaction management system to respond to primitives .
- Practical considerations (performance, consistency requirements, local DBMS capabilities) determine which protocol is appropriate .

G. Real-World Significance

- **Enterprise Application Integration:** Large organizations need to integrate data from multiple legacy systems (ERP, CRM, HR) acquired over decades.
- **E-Commerce:** An order may need to update inventory in one system, process payment in another, and schedule shipping in a third.
- **Healthcare:** Patient records may be distributed across hospital systems, clinics, and insurance databases.

Exam Tips for Unit 10

Topic	Key Focus for Exams
Transactional Workflows	Definition, characteristics, compensation, and the Vinted case study .
E-Commerce Transactions	Traffic spike challenges, memory-first architecture, CQRS, and the Juspay case study .
Real-Time Transactions	Time constraints, deadlines, priority scheduling, and applications.
Long Duration Transactions	Problems with traditional ACID, nested transactions, sagas, compensating transactions.
Multidatabase Systems	Autonomy vs. consistency, 2PC limitations, relaxed ACID, and classification of transactions .

Common Exam Questions:

- Compare and contrast traditional ACID transactions with advanced transaction models.
- Explain how a specific case study (Juspay or Vinted) addresses real-world transaction challenges .
- Discuss the tradeoffs between autonomy and consistency in multidatabase systems .
- Describe how compensating transactions are used in long-duration transactions and sagas.
- What are the limitations of Two-Phase Commit in multidatabase environments?